物件導向程式設計II

姓名:李欣曈 學號:A1143302

HW1-0918-AI 對話紀錄



請問圖片中的程式第一個問題是不是在主程式中少setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);。第二個問題是setVisible(true);放在太前面，應該放在建構子的最後一行。第三個問題是用了setLayout(null);，但沒有手動設定元件的位置。第四個問題是JTextField t1 = new JTextField();沒有設定欄位長度。第五個問題是if (t1.getText() == "admin" && t2.getText() == "，字串不能用==比對。

是的，您觀察得非常仔細，您指出的五個問題完全正確。根據圖片 `{9C8A6361-0831-4B23-B861-8EC57AC8B0CC}.png` 中的程式碼，以下是針對這五個問題的詳細確認：

- **第一個問題正確：** 程式的 `BadLogin` 建構子中確實漏掉了 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`。這會導致使用者點擊視窗右上角關閉時，視窗雖然消失，但 Java 程式仍在背景執行。 PNG
- **第二個問題正確：** `setVisible(true);` 確實放在太前面。它應該放在建構子的最後一行，亦即所有元件（`JLabel`、`JTextField`、`JButton`）都 `add` 完畢之後再呼叫，否則容易出現開啟時視窗一片空白，需要縮放視窗才會顯示元件的 Bug。 PNG
- **第三個問題正確：** 程式中使用了絕對排版 `setLayout(null);`，但是下方所有的元件都直接使用 `add()` 加入，完全沒有呼叫 `.setBounds(x, y, width, height)` 來手動設定位置與大小。這會導致所有元件的大小預設為 0，完全無法在畫面上看見。 PNG
- **第四個問題正確：** `JTextField t1 = new JTextField();` 與 `t2` 確實沒有在括號內設定欄位長度。若搭配其他排版管理員（如 `FlowLayout`），這會導致輸入框預設縮成極小的一點。 PNG
- **第五個問題正確：** 按鈕的事件監聽器中寫了 `t1.getText() == "admin"`。在 Java 中，比對字串「內容」必須使用 `.equals()` 方法（如 `t1.getText().equals("admin")`）。使用 `==` 是比對物件的記憶體位址，這會導致即使輸入正確的帳號密碼，判斷式也永遠為 `false`。 PNG

除了這些問題外，還有需要改的地方嗎?以及我不太確定第五個問題可以這樣修改嗎?btn.addActionListener(e -> {
String username = t1.getText();
String password = new String(t2.getPassword());

if (username.equals("admin") &&
password.equals("1234")) {
System.out.println("登入成功");
} else {
System.out.println("帳號或密碼錯誤");
}

關於您提出的修改，**使用 `.equals()` 來比對字串是完全正確的作法**，且您加上了 `else` 來處理錯誤提示，這讓程式邏輯變得更加完整。

不過，要讓您這段修改能夠順利執行，必須連帶修正圖片中的另一個問題：

1. `t2` **必須改為** `JPasswordField`
在您的修改中，使用了 `t2.getPassword()`，這是實務上獲取密碼最標準且安全的寫法。然而，在原始圖片的程式碼中，`t2` 是被宣告為一般的文字輸入框 `JTextField`。因為 `JTextField` 並沒有 `getPassword()` 這個方法，如果直接套用您的程式碼，會發生編譯錯誤。 PNG

- **修正方式：** 將圖片中的 `JTextField t2 = new JTextField();` 改為密碼專用的輸入框：
`JPasswordField t2 = new JPasswordField();`
這不僅能讓您的 `getPassword()` 順利編譯，還能在使用者輸入時自動將字元隱藏為星號或圓點。 PNG

除了上述與您修改相關的問題外，圖片中的程式碼還有以下可以改進的細節：

2. 啟動介面的執行緒安全 (Thread Safety)
在 Java Swing 中，所有的圖形化元件建立與更新都應該在「事件分派執行緒 (EDT)」上執行，以避免潛在的畫面死結或顯示異常。圖片裡的 `main` 方法直接呼叫 `new BadLogin();`，雖然在這種極小型的程式裡通常會動，但這是不符合規範的寫法。 PNG

- **建議修改：**

```java
public static void main(String[] a) {
    SwingUtilities.invokeLater(() -> {
        new BadLogin();
    });
}
```

3. 缺乏圖形化介面的回饋

目前的程式設計是在終端機印出 `System.out.println("登入成功");`。既然已經撰寫了視窗介面 (GUI)，使用者是不會去看終端機視窗的。

- **建議修改：** 可以將 `System.out.println` 替換為訊息對話框，這樣使用者體驗會更好。

```java
if (username.equals("admin") && password.equals("1234")) {
    JOptionPane.showMessageDialog(null, "登入成功！");
} else {
    JOptionPane.showMessageDialog(null, "帳號或密碼錯誤！", "錯誤", JOptionPane.ERR
}
```

這樣修改可以嗎?import javax.swing.*;
import java.awt.*;

public class HW1 extends JFrame{

這段程式碼修改得非常好，邏輯完全正確，可以直接執行。

你寫的程式碼有幾個很棒的優點：

- **使用** `JPasswordField`：正確使用了密碼專用的文字框，輸入時會顯示隱藏字元（如星號或圓點），符合真實登入介面的需求。
- **正確的字串比較**：使用了 `.equals()` 來比對帳號和密碼，這在 Java 中是正確比對字串內容的方式（沒有誤用 `==`）。
- **Lambda 語法**：使用 `e -> { ... }` 來實作 `ActionListener`，讓程式碼看起來非常簡潔現代。

進階修改建議（讓程式更像真實的 App）：
目前你的登入結果是印在「控制台 (Console)」（`System.out.println`）。如果你希望使用者在圖形介面上直接看到登入成功或失敗的提示，可以把 `System.out.println` 換成 `JOptionPane.showMessageDialog`，這樣會彈出一個提示視窗：

Java

```
btn.addActionListener(e -> {
    String username = t1.getText();
    String password = new String(t2.getPassword());

    if (username.equals("admin") && password.equals("1234")) {
        // 彈出成功視窗
        JOptionPane.showMessageDialog(HW1.this, "登入成功！");
    } else {
        // 彈出錯誤視窗
        JOptionPane.showMessageDialog(HW1.this, "帳號或密碼錯誤！", "錯誤", JOptionPane.
    }
});
```

除此之外，你目前的寫法作為一份作業或基礎練習已經非常標準且完整了！

HW1-0918- Reflection

最一開始投影片裡的程式碼有這些錯誤問題，第一個問題是在主程式中少 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);，所以我增加 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);，因為如果沒有加，會導致視窗看起來是關閉了，Java 程式還在背景執行。第二個問題是 setVisible(true);放在太前面，應該放在建構子的最後一行，因為放太前面會導致視窗開啟時是空白的。第三個問題是用了 setLayout(null);，但沒有手動設定元件的位置，所以我改成自動排版 FlowLayout()。第四個問題是 JTextField t1

= new JTextField();沒有設定欄位長度，所以我預設一個數字，因為沒預設的話，會造成元件變得很小。第五個問題是 if (t1.getText() == "admin" && t2.getText() == "，字串不能用==比對，==比對的是物件記憶體的位址，所以我改用.equals()。第六個問題是用 JTextField t2 = new JTextField();的話，輸入密碼時會直接顯示文字，比較沒安全性，所以我改用 JPasswordField，打出來的密碼才會以小圓點的方式呈現。